BTCS303P - Assignment AS08: SQL (DDL, DML, SELECT, Joins, Subqueries)

Name: Mohd Raza | Roll No: 35 | Section: B | CSE, COER University, Roorkee

Database: college_db (MySQL syntax). Queries were executed in order on the sample data, so outputs reflect the updates in B3/B4, and Part H reflects the committed change in G6.

Part A: DDL (Data Definition Language)

A1. Create a table faculty with columns faculty_id (primary key), name (not null), email (unique), salary and dept_id (foreign key to departments).

Solution

```
CREATE TABLE faculty (
  faculty_id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  salary DECIMAL(10,2),
  dept_id INT,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Explanation

The table is created with the primary key, NOT NULL, UNIQUE and foreign key constraints.

A2. Add a new column phone (VARCHAR(15)) to faculty using ALTER TABLE.

Solution

```
ALTER TABLE faculty ADD COLUMN phone VARCHAR(15);
```

Explanation

A new nullable column phone is appended to faculty.

A3. Rename the column phone to contact_no.

Solution

```
ALTER TABLE faculty RENAME COLUMN phone TO contact_no;
```

Explanation

In older MySQL (5.7) use: ALTER TABLE faculty CHANGE phone contact_no VARCHAR(15);

A4. Drop the column contact_no.

Solution

```
ALTER TABLE faculty DROP COLUMN contact_no;
```

Explanation

The column and all data stored in it are removed.

A5. Remove the faculty table completely, then explain the difference between DROP, TRUNCATE and DELETE.

Solution

```
DROP TABLE faculty;
```

Explanation

DROP removes the whole table: structure, data, indexes and constraints. It cannot be rolled back (DDL).

TRUNCATE removes all rows but keeps the table structure; it is fast, resets AUTO_INCREMENT, takes no WHERE clause, and is DDL (auto-commit, generally not rollback-able).

DELETE removes rows one by one (all or those matching WHERE); it is DML, can be rolled back inside a transaction, fires triggers, and does not reset AUTO_INCREMENT.

Part B: DML (Data Manipulation Language)

B1. Insert a new student: (109, 'Neha', 20, 'Roorkee', 1).

Solution

```
INSERT INTO students VALUES (109,'Neha',20,'Roorkee',1);
SELECT * FROM students WHERE student_id = 109;
```

Output

student_id	name	age	city	dept_id
109	Neha	20	Roorkee	1

B2. Insert a student without specifying city and observe the value stored.

Solution

```
INSERT INTO students (student_id, name, age, dept_id) VALUES (110,'Test',21,1);
SELECT * FROM students WHERE student_id = 110;
```

Output

student_id	name	age	city	dept_id
110	Test	21	NULL	1

Explanation

city has no DEFAULT, so the unspecified column is stored as NULL.

B3. Update the city of student 105 to 'Roorkee'.

Solution

```
UPDATE students SET city = 'Roorkee' WHERE student_id = 105;
SELECT student_id, name, city FROM students WHERE student_id = 105;
```

Output

student_id	name	city
105	Rohan	Roorkee

B4. Increase marks by 5 for all enrollments in course 203.

Solution

```
UPDATE enrollments SET marks = marks + 5 WHERE course_id = 203;
SELECT * FROM enrollments WHERE course_id = 203;
```

Output

student_id	course_id	marks
103	203	86
106	203	79
108	203	71

Explanation

Marks of 103, 106 and 108 rise from 81, 74, 66 to 86, 79, 71. Later outputs in this sheet reflect this change (and B3).

B5. Delete the student with student_id = 109 (and any other row you added for testing).

Solution

```
DELETE FROM students WHERE student_id IN (109, 110);
SELECT COUNT(*) AS total_students FROM students;
```

Output

total_students
8

Explanation

Both test rows are removed; the table is back to 8 students.

B6. Try inserting a student with age 15. Record the error message and explain which constraint caused it.

Solution

```
INSERT INTO students VALUES (111,'Tiny',15,'Roorkee',1);
```

Output

```
ERROR 3819 (HY000): Check constraint 'students_chk_1' is violated.
```

Explanation

Error (MySQL 8.0.16+): ERROR 3819 (HY000): Check constraint 'students_chk_1' is violated.

(PostgreSQL: ERROR: new row for relation "students" violates check constraint "students_age_check".)

The CHECK constraint on age (age >= 17) rejected the row, because 15 < 17. The insert fails and no row is stored.

Part C: Querying with SELECT

C1. Display all columns of all students.

Solution

```
SELECT * FROM students;
```

Output

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
103	Kabir	19	Haridwar	2
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	NULL
108	Isha	19	Roorkee	2

C2. Display only the name and city of all students.

Solution

```
SELECT name, city FROM students;
```

Output

name	city
Aarav	Roorkee
Diya	Dehradun
Kabir	Haridwar
Meera	Roorkee
Rohan	Roorkee
Sana	Dehradun
Vikram	Jaipur
Isha	Roorkee

C3. List students from Roorkee.

Solution

```
SELECT * FROM students WHERE city = 'Roorkee';
```

Output

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
108	Isha	19	Roorkee	2

Explanation

Rohan (105) now appears because B3 moved him to Roorkee.

C4. List students aged between 20 and 22 (use BETWEEN).

Solution

```
SELECT * FROM students WHERE age BETWEEN 20 AND 22;
```

Output

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2

Explanation

BETWEEN is inclusive of both ends (20 and 22).

C5. List students from Roorkee or Dehradun (use IN).

Solution

```
SELECT * FROM students WHERE city IN ('Roorkee','Dehradun');
```

Output

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2
108	Isha	19	Roorkee	2

C6. List students whose name starts with 'A' (use LIKE).

Solution

```
SELECT * FROM students WHERE name LIKE 'A%';
```

Output

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1

C7. List students who have no department assigned (use IS NULL).

Solution

```
SELECT * FROM students WHERE dept_id IS NULL;
```

Output

student_id	name	age	city	dept_id
107	Vikram	23	Jaipur	NULL

Explanation

NULL cannot be compared with =; IS NULL must be used.

C8. Display distinct cities of all students.

Solution

```
SELECT DISTINCT city FROM students;
```

Output

city
Roorkee
Dehradun
Haridwar
Jaipur

C9. Display students sorted by age in descending order, then by name ascending.

Solution

```
SELECT * FROM students ORDER BY age DESC, name ASC;
```

Output

student_id	name	age	city	dept_id
107	Vikram	23	Jaipur	NULL
104	Meera	22	Roorkee	1
102	Diya	21	Dehradun	1
106	Sana	21	Dehradun	2
101	Aarav	20	Roorkee	1
105	Rohan	20	Roorkee	3
108	Isha	19	Roorkee	2
103	Kabir	19	Haridwar	2

C10. Display the top 3 oldest students (LIMIT).

Solution

```
SELECT * FROM students ORDER BY age DESC LIMIT 3;
```

Output

student_id	name	age	city	dept_id
107	Vikram	23	Jaipur	NULL
104	Meera	22	Roorkee	1
102	Diya	21	Dehradun	1

C11. Display name as Student Name and age as Age using aliases.

Solution

```
SELECT name AS `Student Name`, age AS `Age` FROM students;
```

Output

Student Name	Age
Aarav	20
Diya	21
Kabir	19
Meera	22
Rohan	20
Sana	21
Vikram	23
Isha	19

Explanation

Use backticks (MySQL) or double quotes (PostgreSQL) for aliases containing a space.

Part D: Functions and Aggregation

D1. Find the total number of students.

Solution

```
SELECT COUNT(*) AS total_students FROM students;
```

Output

total_students
8

D2. Find the average, minimum and maximum marks in the enrollments table.

Solution

```
SELECT AVG(marks) AS avg_marks, MIN(marks) AS min_marks, MAX(marks) AS max_marks FROM enrollments;
```

Output

avg_marks	min_marks	max_marks
79.27	58	95

D3. Count the number of students in each city.

Solution

```
SELECT city, COUNT(*) AS student_count FROM students GROUP BY city;
```

Output

city	student_count
Dehradun	2
Haridwar	1
Jaipur	1
Roorkee	4

D4. Find the average marks for each course.

Solution

```
SELECT course_id, AVG(marks) AS avg_marks FROM enrollments GROUP BY course_id;
```

Output

course_id	avg_marks
201	83.33
202	82.5
203	78.67
204	58
205	81.5

D5. Show only those cities that have more than one student (HAVING).

Solution

```
SELECT city, COUNT(*) AS student_count FROM students GROUP BY city HAVING COUNT(*) > 1;
```

Output

city	student_count
Dehradun	2
Roorkee	4

D6. Find the total credits offered by each department.

Solution

```
SELECT d.dept_name, SUM(c.credits) AS total_credits
FROM departments d
JOIN courses c ON c.dept_id = d.dept_id
GROUP BY d.dept_name;
```

Output

dept_name	total_credits
CSE	12
ECE	3
Mechanical	3

Explanation

Civil offers no course, so it does not appear (inner join). Use LEFT JOIN to list it with NULL.

D7. Display student names in uppercase and the length of each name (UPPER, LENGTH).

Solution

```
SELECT UPPER(name) AS name_upper, LENGTH(name) AS name_length FROM students;
```

Output

name_upper	name_length
AARAV	5
DIYA	4
KABIR	5
MEERA	5
ROHAN	5
SANA	4
VIKRAM	6
ISHA	4

D8. Explain why WHERE cannot be used with aggregate functions, and what HAVING does differently.

Solution

Explanation

WHERE filters individual rows before grouping and aggregation happen, so aggregate values such as COUNT() or AVG() do not exist yet at that stage and cannot be used in it.

HAVING filters groups after GROUP BY has formed them and aggregates have been computed, so it can use aggregate functions.

Example: SELECT city, COUNT(*) FROM students WHERE age > 19 GROUP BY city HAVING COUNT(*) > 1; - WHERE removes rows first, HAVING then removes small groups.

Part E: Joins

E1. Display each student's name along with their department name (INNER JOIN).

Solution

```
SELECT s.name, d.dept_name
FROM students s
INNER JOIN departments d ON s.dept_id = d.dept_id;
```

Output

name	dept_name
Aarav	CSE
Diya	CSE
Kabir	ECE
Meera	CSE
Rohan	Mechanical
Sana	ECE
Isha	ECE

Explanation

Vikram (no department) is excluded by the inner join.

E2. Display all students, including those without a department (LEFT JOIN). Which student shows NULL?

Solution

```
SELECT s.name, d.dept_name
FROM students s
LEFT JOIN departments d ON s.dept_id = d.dept_id;
```

Output

name	dept_name
Aarav	CSE
Diya	CSE
Kabir	ECE
Meera	CSE
Rohan	Mechanical
Sana	ECE
Vikram	NULL
Isha	ECE

Explanation

Vikram shows NULL for dept_name, since his dept_id is NULL.

E3. Display all departments, including those with no students (RIGHT JOIN or LEFT JOIN with tables swapped).

Solution

```
SELECT d.dept_name, s.name
FROM students s
RIGHT JOIN departments d ON s.dept_id = d.dept_id;
```

Output

dept_name	name
CSE	Aarav
CSE	Diya
ECE	Kabir
CSE	Meera
Mechanical	Rohan
ECE	Sana
ECE	Isha
Civil	NULL

Explanation

Civil has no students, so its name column is NULL. Equivalent: FROM departments d LEFT JOIN students s ON ...

E4. Display student name, course title and marks by joining students, enrollments and courses.

Solution

```
SELECT s.name, c.title, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
```

Output

name	title	marks
Aarav	Data Structures	88
Aarav	Operating Systems	76
Aarav	Machine Learning	91
Diya	Data Structures	67
Diya	Machine Learning	72
Kabir	Digital Electronics	86
Meera	Data Structures	95
Meera	Operating Systems	89
Rohan	Thermodynamics	58
Sana	Digital Electronics	79
Isha	Digital Electronics	71

E5. List the names of students enrolled in 'Data Structures'.

Solution

```
SELECT s.name
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id
WHERE c.title = 'Data Structures';
```

Output

name
Aarav
Diya
Meera

E6. Find students who are not enrolled in any course (LEFT JOIN with IS NULL).

Solution

```
SELECT s.student_id, s.name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

Output

student_id	name
107	Vikram

Explanation

Vikram (107) has no enrollment row.

E7. Display the number of students enrolled in each course (include courses with zero enrollments).

Solution

```
SELECT c.title, COUNT(e.student_id) AS enrolled
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title;
```

Output

title	enrolled
Data Structures	3
Operating Systems	2
Digital Electronics	3
Thermodynamics	1
Machine Learning	2

Explanation

COUNT(e.student_id) counts only non-NULL values, so a course with no enrollments shows 0 (not 1). In this data every course has at least one enrollment.

E8. Display each department with its number of students and its average age.

Solution

```
SELECT d.dept_name, COUNT(s.student_id) AS num_students, AVG(s.age) AS avg_age
FROM departments d
LEFT JOIN students s ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Output

dept_name	num_students	avg_age
CSE	3	21
Civil	0	NULL
ECE	3	19.67
Mechanical	1	20

Explanation

Civil shows 0 students and NULL average age.

Part F: Subqueries

F1. Find students whose age is greater than the average age.

Solution

```
SELECT * FROM students
WHERE age > (SELECT AVG(age) FROM students);
```

Output

student_id	name	age	city	dept_id
102	Diya	21	Dehradun	1
104	Meera	22	Roorkee	1
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	NULL

F2. Find the student(s) with the highest marks in any course.

Solution

```
SELECT s.name, e.course_id, e.marks
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
WHERE e.marks = (SELECT MAX(marks) FROM enrollments);
```

Output

name	course_id	marks
Meera	201	95

F3. List courses that have at least one enrollment (IN or EXISTS).

Solution

```
SELECT * FROM courses
WHERE course_id IN (SELECT course_id FROM enrollments);

-- Same result using EXISTS:
-- SELECT * FROM courses c WHERE EXISTS
--   (SELECT 1 FROM enrollments e WHERE e.course_id = c.course_id);
```

Output

course_id	title	credits	dept_id
201	Data Structures	4	1
202	Operating Systems	4	1
203	Digital Electronics	3	2
204	Thermodynamics	3	3
205	Machine Learning	4	1

F4. List students enrolled in more than 2 courses.

Solution

```
SELECT * FROM students
WHERE student_id IN (
  SELECT student_id FROM enrollments
  GROUP BY student_id HAVING COUNT(*) > 2
);
```

Output

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1

F5. Find the department with the highest number of students.

Solution

```
SELECT d.dept_name, COUNT(*) AS num_students
FROM students s
JOIN departments d ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(*) = (
  SELECT MAX(cnt) FROM (
    SELECT COUNT(*) AS cnt FROM students
    WHERE dept_id IS NOT NULL GROUP BY dept_id
  ) AS t
);
```

Output

dept_name	num_students
CSE	3
ECE	3

Explanation

After B5 and B3, CSE and ECE are tied with 3 students each, so both rows are returned. Rohan moved to Roorkee, not department, so he stays in Mechanical.

F6. For each course, list students who scored above that course's average (correlated subquery).

Solution

```
SELECT e.course_id, s.name, e.marks
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
WHERE e.marks > (
  SELECT AVG(e2.marks) FROM enrollments e2
  WHERE e2.course_id = e.course_id
);
```

Output

course_id	name	marks
201	Aarav	88
205	Aarav	91
203	Kabir	86
201	Meera	95
202	Meera	89
203	Sana	79

Explanation

The inner query depends on e.course_id from the outer query, so it is re-evaluated for each outer row.

Part G: Indexes, Views and Transactions

G1. Create an index on students(city). Run EXPLAIN SELECT * FROM students WHERE city = 'Roorkee'; and note what changes.

Solution

```
CREATE INDEX idx_city ON students(city);
```

Output

```
Index idx_city created.
```

Explanation

Before the index, EXPLAIN shows type = ALL, key = NULL, rows = 8 (full table scan).

After the index, EXPLAIN shows type = ref, possible_keys/key = idx_city, rows = 3 (only matching rows read).

On a table this tiny the optimizer may still choose a full scan, but on large tables the index avoids reading every row.

Commands: EXPLAIN SELECT * FROM students WHERE city = 'Roorkee'; (run before and after CREATE INDEX).

G2. Create a view student_marks that shows student name, course title and marks.

Solution

```
CREATE VIEW student_marks AS
SELECT s.name, c.title, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
```

Explanation

A view is a stored query; it holds no data of its own.

G3. Query the view to list students scoring above 80.

Solution

```
SELECT * FROM student_marks WHERE marks > 80;
```

Output

name	title	marks
Aarav	Data Structures	88
Aarav	Machine Learning	91
Kabir	Digital Electronics	86
Meera	Data Structures	95
Meera	Operating Systems	89

G4. Drop the view and the index.

Solution

```
DROP VIEW student_marks;
DROP INDEX idx_city ON students;
```

Output

```
View student_marks and index idx_city dropped.
```

Explanation

In PostgreSQL use: DROP INDEX idx_city;

G5. Run a transaction: START TRANSACTION;, update marks for student 101 in course 201 to 100, then run ROLLBACK;. Check that the value is unchanged.

Solution

```
START TRANSACTION;
UPDATE enrollments SET marks = 100 WHERE student_id = 101 AND course_id = 201;
SELECT marks FROM enrollments WHERE student_id = 101 AND course_id = 201;  -- 100 inside the transaction
ROLLBACK;
SELECT marks FROM enrollments WHERE student_id = 101 AND course_id = 201;
```

Output

marks
88

Explanation

Inside the transaction the value reads 100; after ROLLBACK it is back to 88, so the change was undone.

G6. Repeat G5 using COMMIT; instead and confirm the change is permanent.

Solution

```
START TRANSACTION;
UPDATE enrollments SET marks = 100 WHERE student_id = 101 AND course_id = 201;
COMMIT;
SELECT marks FROM enrollments WHERE student_id = 101 AND course_id = 201;
```

Output

marks
100

Explanation

After COMMIT the value stays 100, even after reconnecting; the change is durable.

Part H: Challenge Problems (Optional)

H1. Display a rank list of students by their total marks, with ties handled using RANK() or DENSE_RANK().

Solution

```
SELECT s.name, SUM(e.marks) AS total_marks,
       RANK() OVER (ORDER BY SUM(e.marks) DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY SUM(e.marks) DESC) AS dense_rnk
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
ORDER BY total_marks DESC;
```

Output

name	total_marks	rnk	dense_rnk
Aarav	267	1	1
Meera	184	2	2
Diya	139	3	3
Kabir	86	4	4
Sana	79	5	5
Isha	71	6	6
Rohan	58	7	7

Explanation

Window functions need MySQL 8.0+ or PostgreSQL. RANK() leaves gaps after ties; DENSE_RANK() does not.

H2. Assign a grade to each enrollment using CASE: 90 and above = A, 75 to 89 = B, 60 to 74 = C, below 60 = D.

Solution

```
SELECT student_id, course_id, marks,
  CASE
    WHEN marks >= 90 THEN 'A'
    WHEN marks >= 75 THEN 'B'
    WHEN marks >= 60 THEN 'C'
    ELSE 'D'
  END AS grade
FROM enrollments;
```

Output

student_id	course_id	marks	grade
101	201	100	A
101	202	76	B
101	205	91	A
102	201	67	C
102	205	72	C
103	203	86	B
104	201	95	A
104	202	89	B
105	204	58	D
106	203	79	B
108	203	71	C

Explanation

CASE checks conditions top to bottom and returns the first match, so only the lower bound is needed in each branch.

H3. Find pairs of students from the same city (self join).

Solution

```
SELECT a.name AS student1, b.name AS student2, a.city
FROM students a
JOIN students b
  ON a.city = b.city AND a.student_id < b.student_id;
```

Output

student1	student2	city
Aarav	Meera	Roorkee
Aarav	Rohan	Roorkee
Aarav	Isha	Roorkee
Diya	Sana	Dehradun
Meera	Rohan	Roorkee
Meera	Isha	Roorkee
Rohan	Isha	Roorkee

Explanation

a.student_id < b.student_id removes self-pairs and duplicate mirrored pairs.

H4. Find the second highest marks in the enrollments table without using LIMIT.

Solution

```
SELECT MAX(marks) AS second_highest
FROM enrollments
WHERE marks < (SELECT MAX(marks) FROM enrollments);
```

Output

second_highest
95

Explanation

The inner query finds the highest mark; the outer query takes the maximum of everything below it.